

AuraBank - Secure Online Banking System

Software Architecture & Technical Implementation Report | Deliverable Task 2

1. Executive Summary

The **Secure Online Banking System (AuraBank)** is a full-stack, enterprise-ready web application engineered to facilitate secure user authentication, multi-account portfolio management, atomic fund transfers, filterable transaction history, digital receipt exporting, and administrative role-based risk oversight. Built using Node.js/Express, SQLite3, and React 18, the system implements bank-grade security protocols including bcrypt password/PIN hashing, JWT stateless authorization, CORS protections, and real-time security audit logging.

Core Feature Breakdown

Feature	Technical Details	Security / User Value
Secure Login & Registration	JWT Bearer tokens, Bcrypt hash (10 salt rounds)	Prevents credential leakage; automated welcome credit.
Account Management	Checking, Savings (4.5% APY), Investment	Instant account creation & instant freeze/unfreeze action.
Fund Transfer Engine	2-Factor 4-Digit Security PIN validation	Atomic money transfers with zero balance vulnerability.
Transaction History	Real-time search, category filters, receipt export	Searchable audit trail & printable digital receipts.
Role-Based Access Control	Admin Portal & Audit Log Stream	Enables system liquidity monitoring & user account freeze.

2. System Architecture & Tech Stack

The architecture strictly adheres to modern full-stack decoupling principle with a RESTful backend API and a reactive Vite single-page application (SPA).

Layer	Technology Stack	Purpose
Frontend	React 18, Vite, Vanilla CSS, Recharts	Responsive glassmorphic financial portal.
Backend API	Node.js, Express.js, JWT, BcryptJS	RESTful controllers, input validation & auth.
Database	SQLite3 (`online_banking.db`)	Relational integrity with FK foreign key constraints.
PDF Report Engine	Python 3.13, ReportLab Framework	Automated generation of formal project documentation.

3. Database Entity-Relationship & Schema Design

The SQLite database is structured around 5 core tables ensuring strict normalization and foreign key integrity:

- **users:** id, full_name, email, password_hash, transaction_pin, role, status, created_at
- **accounts:** id, user_id, account_number, account_type, balance, status, created_at
- **transactions:** id, reference_id, sender_account_id, receiver_account_id, amount, category, description, status, created_at
- **beneficiaries:** id, user_id, beneficiary_account_number, beneficiary_name, nickname, created_at
- **audit_logs:** id, user_id, action, ip_address, details, created_at

4. REST API Endpoint Reference

HTTP Method	Endpoint Path	Access Level	Description
POST	/api/auth/register	Public	Registers user & creates \$1,000 checking account.
POST	/api/auth/login	Public	Validates credentials & returns JWT token.
GET	/api/accounts	Customer	Fetches user accounts and total balance.
POST	/api/accounts	Customer	Opens savings/checking/investment account.
POST	/api/transfers	Customer	Atomic money transfer with 4-digit PIN verification.
GET	/api/transactions	Customer	Filterable transaction history with search.
GET	/api/admin/analytics	Admin Only	System liquidity, transfer volume & audit logs.
PATCH	/api/admin/users/:id/status	Admin Only	Freezes or activates user access.

5. Security Controls & Defensive Engineering

AuraBank incorporates multiple layers of security to safeguard client capital and sensitive identity records:

- 1. Password & PIN Protection:** All user passwords and 4-digit transaction authorization PINs are salted and hashed using Bcrypt before disk persistence.
- 2. Stateless Session Enforcement:** JWT tokens signed with a secret key expire after 24 hours. Token signatures are validated on every protected API call.
- 3. Atomic Balance Verification:** Fund transfer controller guarantees sender balance sufficiency and updates both sender and recipient accounts within an atomic execution block.
- 4. SQL Injection & XSS Prevention:** All database queries utilize parameterized SQL statements. Input fields strip malicious scripts.
- 5. Immutable Audit Trail:** Sensitive operations (login, transfer, status freeze) write IP address and action details directly to audit_logs.

6. Project Deliverables Matrix

Required Deliverable	Status	Artifact Location
Public GitHub Repository	COMPLETED	Root Directory Structure
Full Application Source Code	COMPLETED	/backend & /frontend

README Documentation	COMPLETED	/README.md
Database Schema (DDL)	COMPLETED	/docs/schema.sql
API Documentation	COMPLETED	/docs/api_documentation.md
Project Report (PDF)	COMPLETED	/docs/Project_Report.pdf